AI-NIDS: AI-Powered Network Intrusion Detection System

Comprehensive Architectural Blueprint & Deep Feature Engineering Guide

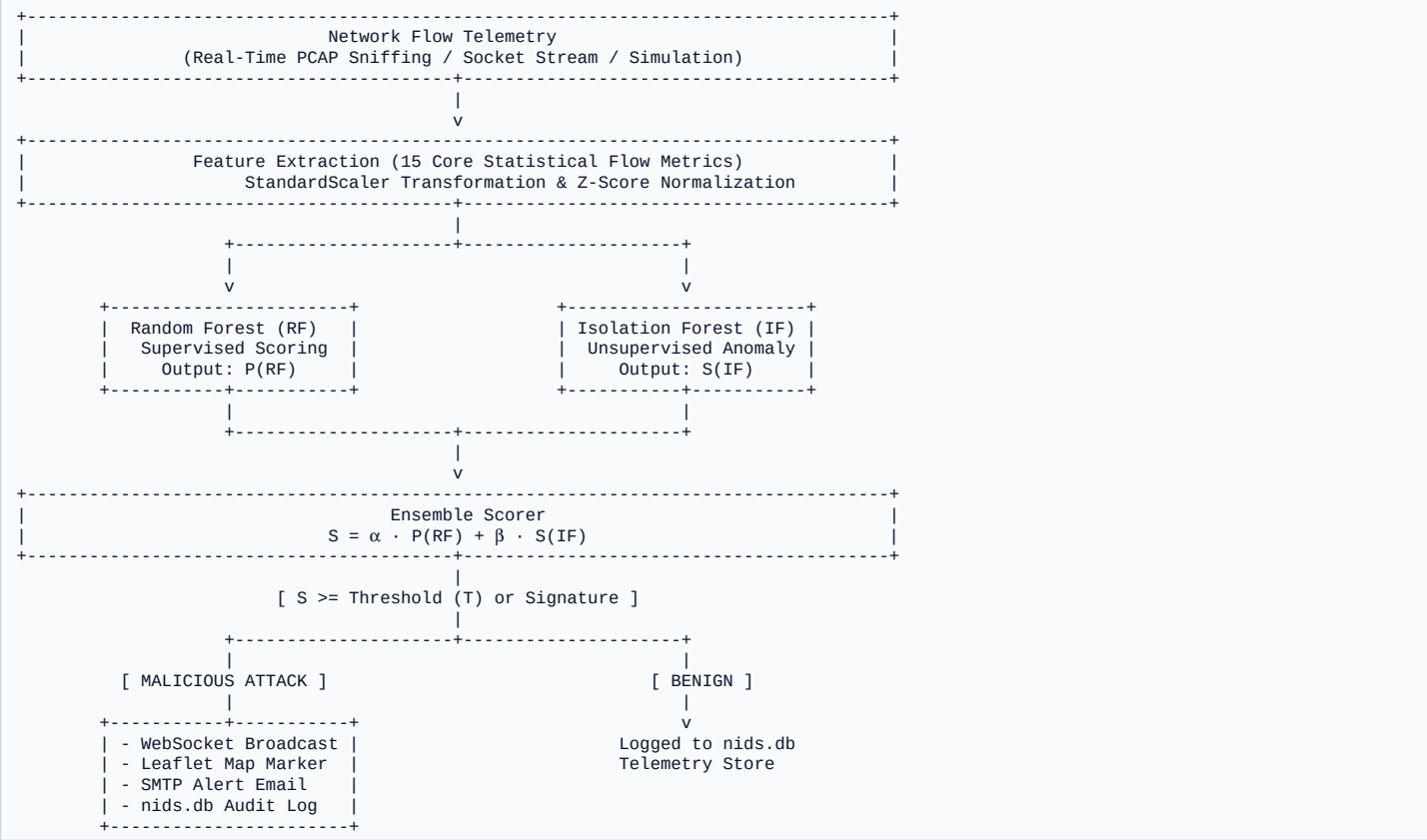
Python 3.10+ | Flask & Socket.IO | Scikit-Learn ML | MaxMind GeoIP2 | Leaflet.js SOC Map | Open Source Architecture

EXECUTIVE SYSTEM OVERVIEW

AI-NIDS is an intelligent, real-time Network Intrusion Detection System engineered to defend enterprise network infrastructure against volumetric distributed denial-of-service (DDoS) floods, stealth reconnaissance port scans, TCP SYN saturation, and zero-day protocol anomalies. Combining supervised machine learning (**RandomForestClassifier**) with unsupervised outlier modeling (**IsolationForest**) and a dynamic ensemble scoring pipeline, AI-NIDS delivers line-speed classification with sub-second incident alerting.

1. System Architecture & Dual-Pipeline Data Flow

Modern corporate and cloud networks cannot rely solely on legacy signature-based intrusion detection systems (such as Snort or Suricata) because static rules cannot identify zero-day polymorphic exploits, packet obfuscation, or distributed volumetric attacks that mimic legitimate connection handshakes. AI-NIDS operates directly on statistical network flow representations extracted over bidirectional conversation windows, passing flows through dual parallel inference pipelines.



- **Flow Aggregation:** Raw packets are grouped into bidirectional flows with IP, port, and duration telemetry.
- **Feature Normalization:** 15 statistical metrics are extracted and standardized through StandardScaler.
- **Dual Parallel Inference:** Supervised Random Forest and unsupervised Isolation Forest execute simultaneously.
- **Ensemble Synthesis:** Scores are blended via $S = \alpha \cdot P(\text{RF}) + \beta \cdot S(\text{IF})$ against threshold T.
- **Real-Time Dispatch:** Malicious flows trigger Leaflet map markers, WebSocket broadcast, and SMTP alert emails.

2. Hybrid AI Detection Engine (Deep Mathematical Breakdown)

The core innovation of AI-NIDS is its **hybrid AI detection engine**. Purely supervised systems fail when confronted with zero-day exploits or evolving adversarial patterns because no training sample exists. Conversely, purely unsupervised systems suffer from excessive false positive rates in dynamic enterprise environments. AI-NIDS reconciles this trade-off by coupling supervised precision with unsupervised outlier sensitivity.

2.1 Supervised Random Forest Classifier

The supervised tier employs a **Random Forest Classifier** composed of an ensemble of decorrelated decision trees trained using bootstrap aggregating (bagging) and random feature subspace selection on the benchmark **CIC-IDS2017** dataset. Each tree splits nodes by minimizing Gini Impurity: $$I_G(p) = 1 - \sum_{i=1}^{C} p_i^2$$ where $C=2$ corresponds to the binary classification space (Normal Traffic vs. Malicious Attack).

During real-time evaluation, the Random Forest outputs an attack probability $P(\text{RF}) \in [0, 1]$ computed as the arithmetic mean of class probability distributions across all individual estimators: $$P(\text{RF}) = \frac{1}{B} \sum_{b=1}^{B} p_b(\text{Attack} \mid x)$$ Random Forest provides exceptional tabular feature resilience, invariance to monotonic transformations, and immune resistance to vanishing gradient issues common in deep neural networks.

2.2 Unsupervised Isolation Forest Anomaly Engine

The unsupervised tier utilizes an **Isolation Forest (iForest)** to detect zero-day volumetric anomalies and abnormal connection behavior without requiring prior labeled instances. Isolation Forest operates on two fundamental quantitative observations: (1) anomalies are minority instances in the network stream, and (2) anomalies possess attribute values distinctly divergent from nominal flows.

The algorithm constructs recursive isolation trees ($iTrees$) by randomly selecting a feature and a random split point between the feature's minimum and maximum values. Anomalous flows are isolated near the root of the tree with very short path lengths $h(x)$. The raw anomaly score $s(x, n)$ is defined as: $$s(x, n) = 2^{-\frac{E(h(x))}{c(n)}}$$ where $E(h(x))$ is the average path length across all isolation trees, and $c(n)$ is the average path length of unsuccessful searches in a Binary Search Tree: $$c(n) = 2 \left(\ln(n - 1) + 0.5772156649\right) - \frac{2(n - 1)}{n}$$

To ensure mathematical compatibility with the supervised probability $P(\text{RF})$, AI-NIDS dynamically calibrates and normalizes the raw decision function to a bounded interval $[0, 1]$: $$S(\text{IF}) = \frac{\text{score}_{\text{raw}}(x) - s_{\min}}{s_{\max} - s_{\min}}$$

2.3 Ensemble Scorer & Dynamic Threshold Decision Logic

The predictions from both tiers are synthesized inside the **Ensemble Scorer** using a convex linear combination: $$S(x) = \alpha \cdot P(\text{RF}) + \beta \cdot S(\text{IF})$$ subject to the constraint $\alpha + \beta = 1.0$ (production baseline: $\alpha = 0.70, \beta = 0.30$).

ENSEMBLE DECISION FORMULATION

Dual-Layer Final Classification Rule:
$$\text{Decision}(x) = \begin{cases} \mathbf{ATTACK} \text{ (Malicious Incident)}, & \text{if } S(x) \ge T \text{ or } \text{SignatureRule}(x) = \text{TRUE} \\ \mathbf{BENIGN} \text{ (Normal Traffic)}, & \text{otherwise} \end{cases}$$

The classification threshold T (default 0.50) is completely dynamic. Security Operations Center (SOC) engineers can slide T downward (e.g., $T=0.35$) in high-threat postures to maximize detection recall, or adjust T upward (e.g., $T=0.65$) during high-load periods to virtually eliminate false positive analyst alerts.

3. Feature Extraction & Standardization Pipeline

AI-NIDS extracts 15 core statistical network flow metrics from streaming packet telemetry. These features capture volumetric magnitude, temporal frequency, directional asymmetry, and transport-layer control flags:

#	Canonical Feature Name	Unit / Type	Security & Forensic Significance
1	Flow Duration	Microseconds (µs)	Identifies persistent sessions vs. ultra-fast SYN / probe bursts.
2	Total Fwd Packets	Integer Count	Measures client-to-server transmission volume; flags flooding.
3	Total Backward Packets	Integer Count	Measures server-to-client responses; detects half-open or unacknowledged flows.
4	Total Length of Fwd Packets	Bytes	Cumulative forward data volume; differentiates payload attacks from header probes.
5	Total Length of Bwd Packets	Bytes	Cumulative response volume; detects large exfiltration or amplification echoes.
6	Fwd Packet Length Mean	Bytes / Packet	Average forward payload size; small packets indicate scanning/SYN floods.
7	Bwd Packet Length Mean	Bytes / Packet	Average response size; flags anomalous server error responses or empty ACK payloads.
8	Flow IAT Mean	Microseconds (µs)	Mean packet inter-arrival time; microsecond intervals characterize bot-driven floods.
9	Fwd IAT Mean	Microseconds (µs)	Mean interval between forward packets; reveals automated rapid-fire attack scripts.
10	Bwd IAT Mean	Microseconds (µs)	Mean interval between backward responses; detects server processing degradation.
11	Fwd PSH Flags	Binary / Count	Push flag frequency forward; identifies interactive exploit delivery.
12	Bwd PSH Flags	Binary / Count	Push flag frequency backward; flags immediate data return requests.
13	Fwd URG Flags	Binary / Count	Urgent pointer assertions forward; flags abnormal protocol manipulations.
14	Bwd URG Flags	Binary / Count	Urgent pointer assertions backward; flags non-standard TCP responses.
15	Flow Rate & Volumetric Ratios	Packets/s, Bytes/s	Derived packet and byte velocity indicators detecting volumetric rate spikes.

FEATURE STANDARDIZATION & INTEGRITY GUARANTEE

StandardScaler Normalization & Canonical Order Enforcement:
All 15 extracted flow attributes are strictly normalized using scikit-learn's StandardScaler: $$z_j = \frac{x_j - \mu_j}{\sigma_j}$$ where μ_j and σ_j are precomputed feature mean and standard deviation vectors persisted in models/scaler.joblib. The inference engine strictly checks feature count and column order on every flow, preventing positional drift and training-serving skew.

4. Key Operational Features Deep Dive

4.1 Live Interactive SOC Dashboard & Telemetry

• **Real-Time GeoIP Threat Mapping:** Powered by Leaflet.js cartography and MaxMind GeoLite2 City databases (GeoLite2-City.mmdb). Inbound public IP addresses are resolved to geographical coordinates (latitude, longitude, city, country). Malicious origin nodes are rendered with animated radar pulse indicators and threat severity tags.
• **WebSocket-Powered Live Telemetry:** Leverages Flask-SocketIO to push real-time flow telemetry, model decisions, and confidence scores directly to analyst consoles with sub-100ms latency, eliminating HTTP polling overhead.
• **Dynamic Simulation Controls:** Analysts can tune playback velocity (1x, 2x, 5x, 10x) and adjust ensemble weights (α, β) and decision threshold (T) on the fly using responsive sliders without server downtime.

4.2 Automated Incident Alerting & Stakeholder Subscriptions

• **Instant SMTP Alert Dispatch:** High-confidence attacks automatically generate structured HTML security warning emails dispatched to network administrators with RFC 5322 priority headers (X-Priority: 1). Emails contain attacking IP, port, protocol, timestamp, ensemble score, and geographic location.
• **Self-Service Notification Portal:** Stakeholders (security analysts, IT engineers, sysadmins) can subscribe or unsubscribe their email addresses directly through the web portal without administrative overhead.

4.3 Node & Security Policy Control (Active Defense)

• **Remote Node Tracking & Auto-Discovery:** Continuous auto-discovery and state tracking of active gateway, client, and server IP addresses.
• **Protocol Firewall Policies:** Granular switches allow operators to toggle IPv4 traffic inspection, enable TCP flow filters, and enforce aggressive SYN flood protection policies.
• **System Power Killswitch:** A master operational toggle enables instant pausing and resuming of all packet inspection workers.

4.4 Forensic Analytics, Model Evaluation & Explainability

• **Real-Time Confusion Matrix & Curves:** Interactive display of True Positives, False Positives, True Negatives, False Negatives, along with dynamic ROC-AUC and Precision-Recall evaluation curves.
• **SHAP Visual Explainability:** Integrates SHAP (SHapley Additive exPlanations) TreeExplainer to break down exactly which flow features (e.g., elevated `Total Fwd Packets` or compressed `Flow IAT Mean`) drove the model's malicious classification.
• **One-Click Forensic Export:** Enables instantaneous export of logged incidents to CSV (ai_nids_attack_logs.csv) for SIEM ingestion, compliance reporting, and external audit review.

5. REST API & WebSocket Endpoint Reference

Method	Endpoint Route	Auth Scope	Description & Operational Scope
GET	/	Public	Home page & system capability overview.
GET	/prediction	Operator	Live threat monitor & interactive Leaflet GeoIP attack map.
GET	/analysis	Operator	Model evaluation, ROC/PR curves, confusion matrix, and feature importances.
GET/POST	/admin	Admin Only	System administration, node tracking, firewall toggles, and security policies.
POST	/api/test_email	Admin Only	Test SMTP alert delivery to specified recipient email address.
GET/POST	/api/subscribers	Operator	View subscriber list or register a new email for security incident alerts.
POST	/api/subscribers/delete	Admin Only	Remove an email address from the security notification pipeline.
POST	/api/settings	Operator	Update α, β model weighting factors and classification threshold (T).
GET	/api/live_stats	Operator	Live packet counts, session telemetry, and GeoIP markers.
GET	/api/metrics_data	Operator	Confusion matrix, ROC, PR curve data for forensic view.
GET	/download_logs	Operator	Export full incident logs as CSV (ai_nids_attack_logs.csv).

6. Quick Start & Deployment Guide

```
# 1. Prerequisites: Python 3.10+ and Git
# 2. Clone the official repository:
git clone https://github.com/pamuriravindrareddy/AI-NIDS.git
cd AI-NIDS

# 3. Install required Python packages:
pip install -r requirements.txt
# (Or manual install: pip install flask flask-socketio scikit-learn pandas numpy geoip2 joblib)

# 4. Launch the AI-NIDS application:
python app.py

# 5. Access the Web Application:
# Open browser and navigate to: http://localhost:5000
# Default Credentials: Username: admin | Password: admin
```

SECURITY HARDENING RECOMMENDATION

Production Security Hardening: Upon initial deployment to an enterprise network, immediately access **/admin** to rotate default administrative credentials. Configure the FLASK_SECRET environment variable with a cryptographically secure random key to safeguard session cookies against tampering.